

My Reflection on Text Analytics: Learning Tokenization, TF-IDF, and Word Embeddings for the Capstone Project

Ram Kelly Jersen Leliza
College of Computer Studies and
Engineering
Jose Rizal University
Manila City, Philippines
ramkellyjersen.leliza@my.jru.edu

Abstract—This paper is my reflection on the Unit 5 text analytics activity for ITC-C508. In this activity, I learned about tokenization, word frequency, TF-IDF, and word embeddings, and I tried them out using two simple web tools: Chat Playground and Language Playground. As a fourth-year student who is about to start the capstone project, I used this paper to connect what I learned in this small activity to what I will need for my future subjects and for my capstone system, Be@Mandaluyong / CCAT 4SIGHT. I talk about what I observed while doing the lab, the main lessons I got from it, and what I still want to learn about NLP and LLMs.

Keywords—tokenization, TF-IDF, word embeddings, semantic vectors, PII extraction, text analytics, natural language processing

I. INTRODUCTION

Text analytics is part of AI, under natural language processing (NLP), and it is about getting meaning out of text that has no fixed structure, like reviews, messages, or documents. Before this activity, most of what I knew about NLP came from our past machine learning lessons — things like classification models, bag-of-words features, and the idea that language models guess the next word. But I never really saw, step by step, how a plain text turns into tokens, then into numbers, then into a summary, a detected language, or a hidden (redacted) personal detail.

This is what the Unit 5 lab helped me understand better. It combined short readings about tokenization, frequency analysis, TF-IDF, and embeddings with an actual hands-on activity using two browser tools: the Chat Playground, which runs a small AI model called Phi-3.5-mini, and the Language Playground, which uses simpler rule-based and statistical methods to detect language and to find personal information (PII) in text. In this paper, I will share what I noticed while doing the lab, the lessons I learned, and how these connect to my coming subjects and to my capstone.

II. EXPERIENTIAL AND PRACTICAL INSIGHTS

A. Using AI to Summarize Text

In the Chat Playground, I set the instructions so the AI would act as an assistant that reads and summarizes text. I then gave it an old magazine review about the Commodore 64 computer and asked it to summarize the review. The AI was able to shrink the whole review into one short paragraph, and it kept both the good points (the price, the

sound chip, the graphics) and the bad points (the keyboard feel, the weak documentation, and the extra cost of accessories) mentioned in the original text. This showed me the difference between extractive and abstractive summarizing: the AI did not just copy sentences, it wrote the summary in its own words, which is only possible because it understands the meaning of the text and not just the word count.

B. Using Rule-Based Tools for Analysis

The Language Playground activity taught me more about the limits of these tools. When I tried the language detection feature on a Japanese sample text, it only gave an 83 percent confidence score instead of being fully sure, which reminded me that even a working model is only giving its best guess, not a perfect answer. I also tried it on a short German product label (from an old Amstrad computer), and this showed that the amount of text really affects how confident the detection is. When I used the PII extraction feature on a sample invoice, it correctly found the customer's name, address, and phone number. But the tool also mentioned that it uses a mix of AI, statistics, and pattern matching, and that it can make mistakes — either missing real personal information or wrongly flagging something that is not. This was a good reminder that these small browser-based tools are only for learning, not for real data privacy work.

Doing these two activities helped me really feel the difference between a fixed, rule-based way of processing text and a more flexible, AI-based way — something I only understood in theory before this.

III. CORE LEARNING OUTCOMES

A. Different Ways to Tokenize Text

Turning text into tokens is the first step before doing anything else with it, but the lab showed me that there is more than one way to do it. Making everything lowercase and removing punctuation (normalization) helps with counting word frequency, but it can also remove important meaning — like the difference between the name "Banks" and the plain word "banks." Removing common words like "the" or "a" (stop words) helps highlight the important words, but assumes those removed words really add nothing. Stemming is quick but can group unrelated words together by mistake, while lemmatization is more accurate but takes more processing power. So basically, every

method has a trade-off between speed, accuracy, and how easy the results are to understand — and the best choice depends on what the text will be used for.

B. TF-IDF vs. Word Embeddings

Simple word counting and TF-IDF are cheap and useful ways to find out what a document is about or how documents are different from each other. But the sample comparison in the lab showed a clear weakness: when two documents share a lot of the same words — like "agent," "AI," and "Microsoft" showing up in both a Copilot Studio text and a Microsoft Foundry text — TF-IDF gives those words a score of zero, meaning they are no longer useful for telling the documents apart. Only the less common words, like "declarative" and "Copilot Studio" versus "code" and "Foundry," actually stood out. Word embeddings do not have this problem because they compare the actual meaning of the words, not just how many times they appear. The example comparing "dog," "cat," and "tree" using cosine similarity, and the vector math showing that "dog" plus "young" equals "puppy," made it clear why embeddings work better than simple counting when meaning matters.

C. Context-Based Vectors and Sentence Ranking

Older embedding methods like Word2Vec or GloVe give one fixed vector to a word no matter the context it is used in. What I read about attention, and how it creates the context-aware embeddings used in GPT-style models, helped me understand why newer AI models handle words with multiple meanings so much better — the meaning of a word changes depending on the other words around it in that specific sentence, instead of being fixed forever. I also learned about TextRank, which ranks sentences the same way Google ranks web pages, and picks the most connected sentences as a summary without needing a generative model at all. This gave me a simpler, non-AI option to compare against the Chat Playground's approach.

IV. COURSE APPLICATION AND IMPACT

These basic concepts will really matter for the subjects and milestones I still have left. Our capstone, Be@Mandaluyong / CCAT 4SIGHT, already plans to have a sentiment analysis feature for tourist feedback, and this lab gave me the right words and ideas to explain the preprocessing choices — like tokenization, removing stop words, and choosing between stemming and lemmatization — instead of just treating that part as a black box.

The TF-IDF versus embeddings comparison is also useful for any search feature we might add to the tourism system: TF-IDF-based keyword search over the attraction descriptions is fast and easy to explain, but depends a lot on the exact words used, while embedding-based search would understand different ways tourists might phrase the same question. I could start with TF-IDF as a simple baseline and later try embeddings for a better search experience.

The PII extraction activity also connects to the data privacy requirements our capstone will need, especially for the user registration and feedback sections. Knowing that even a working PII detector can make mistakes tells me that any similar feature I build should still have a human check the results, or use a more reliable service, instead of trusting one automatic pass alone. Lastly, comparing the small Phi-3.5-mini model running in the browser to a bigger cloud-based model gave me an idea I will need again soon: choosing between a small, fast model and a bigger, more accurate one, which is the same kind of decision I already faced while building my fraud detection and churn prediction models.

V. CONCLUSION AND FUTURE WORK

This activity helped me move from just knowing the names of NLP techniques to actually understanding how they work together: tokenization feeds into frequency counting and embeddings, TF-IDF is fast but depends on the exact words used, and embeddings understand meaning better but cost more to compute. Trying both a generative summarizing tool and a rule-based extraction tool made these trade-offs feel real instead of just something I read in a textbook.

Going forward, I want to study more about how attention and transformers work, since they are the reason behind both the embeddings I learned here and the good summaries the Chat Playground produced. I also want to look into retrieval-augmented generation (RAG), which combines the simplicity of TF-IDF-style search with the better understanding that comes from embeddings, since this seems like the most likely setup I will use if I add a chatbot or search feature to my capstone project.